



Bahria University, Islamabad

Department of Software Engineering

Data Structures & Algorithms Lab (Spring-2026)

Teacher: ENGR. FAISAL ZIA

Student: Khusro Khan

Daniyal Ahmed

Enrollment: 09-131242-062

09-131242-022

Project Report

Date: 26/05/2026

Comments:

Signature

Abstract

This project implements a Blood Donation Management System with two integrated layers: (1) a C++ console application built on five core data structures — Linked List, Queue, Stack, Binary Search Tree (BST), and Graph and (2) a Node.js/Express REST API backend paired with an HTML/CSS/JavaScript web dashboard. The console layer handles donor registration, request queuing, donation history, inventory management, and inter-bank routing with full file-based persistence. The web frontend extends this with a browser-based dashboard, role-based authentication (admin/patient), real-time stock display, CNIC-based donor records, health condition tracking, and Haversine-based GPS bank routing all sharing the same pipe-delimited flat-file storage.

1. Introduction

1.1 Problem Statement

Manual blood bank systems suffer from slow donor–patient matching, error-prone records, no real-time inventory visibility, and no mechanism to locate nearby banks in emergencies. The WHO estimates a global need of 117.4 million blood units annually, making efficient digital management systems critical. This project bridges the gap by providing both a structured C++ DSA implementation and a usable web interface for real-world deployment.

1.2 Objectives

- Manage donors dynamically (with CNIC, health condition, city, contact) using a Linked List.
- Process blood requests fairly (FIFO) using a Queue with atomic stock deduction.
- Track donation history in reverse-chronological order using a Stack.
- Maintain and search blood inventory in real-time using a Binary Search Tree.
- Model blood bank connectivity and compute nearest bank via adjacency matrix (console) and Haversine formula (web).
- Persist all data via pipe-delimited flat files shared between both application layers.

2. Literature Review

Cormen et al. (2009) establish the theoretical basis for BST operations with $O(\log n)$ average-case search, directly applied to blood stock lookups in this system. Sedgewick and Wayne (2011) demonstrate adjacency matrix representations for weighted graphs, implemented in the console layer's inter-bank routing module. Weiss (2013) formalizes Stack (LIFO) and Queue (FIFO) structures as the natural choice for history logs and fair request queuing respectively.

Garg et al. (2014) proposed a PHP/MySQL web-based blood bank system supporting online registration but without in-memory DSA optimization; this project addresses that gap by embedding data structures in the core logic. Kaur and Sharma (2017) described a GPS-enabled mobile app for donor location an approach extended here using the Haversine formula in the Node.js backend for real GPS distance computation between banks. The AABB Technical Manual (2020) defines the ABO/Rh compatibility rules implemented in both layers' compatibility checker. Pressman (2014) advocates boundary-value analysis for healthcare input validation, reflected in the six custom validators in the C++ layer.

3. System Architecture

3.1 Dual-Layer Design

The system is built in two coupled layers that share the same flat-file storage:

Layer 1: C++ Console Application Visual Studio solution (blooddonationsystem.sln). The main class BloodDonationSystem composes five DSA classes. Menus are rendered with Unicode boxdrawing characters. Data is read on startup and written on exit via loadAllData() / saveAllData().

Layer 2: Node.js/Express Web Application server.js exposes a REST API consumed by a single-page HTML/CSS/JS frontend (index.html, styles.css, app.js). , Haversine-based GPS routing. Reads and writes the same pipe-delimited .txt files as the C++ layer.

3.2 Class & Module Structure

Class / Module	Layer	Data Structure	Responsibility
DonorList	C++	Singly Linked List	CRUD on donor records; bubble sort by name
RequestQueue	C++	FIFO Queue	Enqueue/dequeue requests; atomic stock check
HistoryStack	C++	LIFO Stack	Push/pop donation records; LIFO display
BloodStockBST	C++	Binary Search Tree	Insert, search, update, consume blood units
BloodBankGraph	C++	Adjacency Matrix Graph	Model bank network; find nearest bank ($O(V)$)
Express REST API	Node.js	File I/O + JSON	CRUD endpoints; process requests; stock updates
Web Frontend	HTML/JS	DOM / Fetch API	Dashboard, donor/request/stock/bank UI

4. Data Structures & Implementation

4.1 Singly Linked List — Donor Management

New donors are prepended at the head ($O(1)$). Deletion re-links pointers in $O(n)$. Blood-group search traverses all nodes. Bubble sort operates on node data. The Donor struct includes: name, age, bloodGroup, city, contact, lastDonationDate, and (in the web layer) disease and CNIC. Dynamic allocation means no capacity limit.

4.2 FIFO Queue — Request Processing

Requests are enqueued at the rear and dequeued from the front ($O(1)$), ensuring fairness. `processNext()` atomically calls `BloodStockBST::consumeUnits()` before removing the front request, preventing partial fulfilment when stock is insufficient. The web layer mirrors this logic in the POST `/api/requests/process` endpoint.

4.3 LIFO Stack — Donation History

Each `push()` prepends a new `HistoryRecord` to the top ($O(1)$). Displaying history traverses top-to-bottom, naturally yielding most-recent-first order the expected behavior for audit logs. Records include donor name, CNIC (web layer), blood group, and date.

4.4 Binary Search Tree — Blood Stock

Blood group strings serve as BST keys (lexicographic). With 8 fixed groups, tree depth is bounded at ~ 3 levels, giving effectively $O(1)$ search. In-order traversal displays sorted stock. `consumeUnits()` checks availability before decrementing, preventing negative stock.

4.5 Adjacency Matrix Graph — Blood Bank Network

Banks are vertices (up to 10); distances (km) are edge weights in a 10×10 integer matrix. `findClosestBank()` scans a bank's row for the minimum non-zero weight in $O(V)$. The web layer replaces this with real GPS coordinates (latitude/longitude) and Haversine distance computation, returning the geographically nearest bank via GET `/api/banks/closest`.

Data Structure	Key Operations	Time Complexity
Linked List	Insert, Delete, Search, Sort	$O(1)$ insert · $O(n)$ others

Queue	Enqueue, Dequeue, Process	$O(1)$ all
Stack	Push, Pop, Display	$O(1)$ push/pop · $O(n)$ display
BST	Insert, Search, Update, Consume	$O(\log n)$ avg (bounded $\sim O(1)$)
Graph (Console)	Add Bank, Add Route, Find Closest	$O(1)$ edge lookup · $O(V)$ search
Graph (Web)	Haversine GPS routing	$O(B^2)$ route build · $O(B)$ closest

5. Key Features, Validation & File Persistence

5.1 Web Frontend & REST API

- Dashboard with live summary cards: donor count, pending requests, history count, and pergroup stock.
- Donor form includes: Name, Age, Blood Group, City, Contact, CNIC, Last Donation Date, Health Condition.
- Request form includes: Patient Name, Blood Group, Units, Hospital, Patient Health, Timestamp.
- GET /api/banks/closest returns the nearest bank by real GPS Haversine distance.

5.2 Input Validation (C++ Layer)

Validator	Rule
isValidName()	2–50 chars; letters and spaces only
isValidAge()	Integer 18–65 (WHO donor eligibility)
isValidBloodGroup()	Must be one of: A+, A–, B+, B–, O+, O–, AB+, AB–
isValidContact()	10–15 chars; digits, +, –, spaces allowed
isValidDate()	DD/MM/YYYY or DD-MM-YYYY format, exactly 10 chars
isValidUnits()	Integer 1–100

5.3 File Persistence & Compatibility

Both layers share four pipe-delimited (|) flat files: donors.txt (name|age|bloodGroup|city|contact|date|disease|cnic), requests.txt (patient|bg|units|hospital|health|timestamp), history.txt (donor|cnic|bg|date), blood_stock.txt (bg|units). The web layer also uses banks.json (GPS coordinates) and users.json (credentials). On startup the C++ app calls loadAllData(); on exit, saveAllData(). The web server reads files per request with no caching. normalizeBloodGroup() (toUpper + trim) ensures consistent keys across both layers. getCompatibleDonorGroups() implements full ABO/Rh rules (e.g. AB+ accepts from all 8 groups; O– only from O–).

6. Testing

ID	Layer	Module	Input / Action	Expected	Result
TC01	C++	Donor	Age = 17	Error: must be 18–65	✓ PASS
TC02	C++	Donor	Blood Group = 'XY+'	Error: invalid group	✓ PASS

TC03	C++	Stock	Consume 200 units O+	Error: insufficient stock	✓ PASS
TC04	C++	Request	Process: stock=10, need=5	Fulfilled; stock → 5	✓ PASS
TC05	C++	History	Push A,B,C; display	C, B, A (LIFO)	✓ PASS
TC06	C++	Graph	Closest to General Hospital	Red Cross Center (3 km)	✓ PASS
TC07	Web	API	POST /api/requests/process (low stock)	HTTP 400: insufficient stock	✓ PASS
TC08	Web	Banks	GET /api/banks/closest?bank=DHQ	Returns nearest GPS bank + km	✓ PASS
TC09	Both	File I/O	Save data → restart → load	All records restored intact	✓ PASS

7. Conclusion

The Blood Donation Management System successfully integrates five fundamental data structures in a C++ console application and extends it with a full-stack Node.js/Express REST API and HTML/CSS/JS web dashboard. All 10 test cases pass. The dual-layer architecture demonstrates $O(1)$ queue and stack operations, $O(\log n)$ BST stock management, and real GPSbased routing in the web layer within a robust and persistent system. Planned enhancements include a Qt native GUI, SQLite backend, Dijkstra's optimal routing in the console layer, and mobile app integration.